System & Database Workflow

60-80HP Line - Cycle Time Monitoring

Comprehensive documentation of system architecture, signal flow, database tables, and API endpoints.

Flask + MySQL + Arduino UNO + 9 Tablets

1. System Architecture Overview

Hardware Layer

```
Arduino UNO #1 (Material Setter)        Arduino UNO #2 (Cycle Time)
  Pin 4  : IR Sensor 1 (ID detection)     9 START/STOP buttons
  Pin 5  : Relay 2 OUT (solenoid)          for Process 1-9
  Pin 14 : IR Sensor 2 (dunpla at P1)

Zebra Printer: Prints Kitting QR codes via ZPL/CPCL
```

Software Layer (Windows PC)

```
arduino_bridge_ms.py    <-- Bidirectional serial bridge (MS Arduino)
arduino_bridge.py       <-- Read-only serial bridge (CT Arduino)
         |                          |
         +--- HTTP POST ---+--- HTTP POST ---+
                           |
              cycleTimeMoni.py (Flask :5001)
                           |
              database_manager.py --> MySQL (XAMPP)
              qr_printer.py       --> Zebra Printer
              torisql.py          --> Toritani External DB
```

Client Layer (Wi-Fi Network)

9 Tablets (Process 1-9): QR scan, timer, PASS/NG controls.
Material Setter Tablet/PC: Job Order, materials, kitting QR, operator IN/OUT.
Monitoring: Cycle Graph, Line Trend pages.

2. Signal & Data Flow

2A. Material Setter Flow

Step 1: Operator Arrives

IR Sensor (Pin 4) detects -> Arduino sends 'MS1LOAD' -> Bridge POSTs to /api/ms_arduino_signal
-> Flask sets ms1_loaded=True -> Material Setter polls /api/ms_button_status (500ms)
-> Auto-opens IN modal, focuses scan input -> Operator scans ID with handheld scanner
-> POST /api/ms_operator_in -> saved to mtrl_set_operator table

Step 2: Prepare Materials

Operator scans Job Order -> fetches from Toritani DB
Operator scans materials per row -> saved to MAIN_DB
Operator scans Kitting QR code -> POST /api/complete_kitting
-> Saves to KITTING_DB + 26 material tables + joborder_plan
-> Flask queues 'RELAY2_ON' -> Bridge polls /api/ms_get_pending_command
-> Bridge writes 'RELAY2_ON' to Arduino serial
-> Arduino Pin 5 HIGH -> Relay ON -> Solenoid OPENS
-> Arduino confirms 'MS_OPEN_INTERLOCK' -> Badge shows 'INTERLOCK OPEN' (green)

Step 3: Dunpla Taken to Process 1

IR Sensor (Pin 14) detects -> Arduino sends 'P1START_MS'
-> Flask queues 'RELAY2_OFF' -> Bridge sends to Arduino
-> Pin 5 LOW -> Relay OFF -> Solenoid CLOSES
-> Arduino confirms 'MS_CLOSE_INTERLOCK' -> Badge shows 'INTERLOCK CLOSED' (red)

Step 4: Operator Leaves

IR Sensor (Pin 4) clears -> Arduino sends 'MS1UNLOAD'
-> Flask sets ms1_loaded=False -> POST /api/ms_operator_out -> Operator signed out

2B. Cycle Time Flow

Step 1: QR Scanned at Tablet

Tablet scans QR -> POST /api/validate_kitting_qr
-> Validates against KITTING_DB, checks sequence, checks prev process done

Step 2: Start Timer

Operator presses START -> POST /api/start_process -> Server-side timer starts, counter increments

Step 3: Stop Timer

PASS: POST /api/stop_process -> saves to process_{N} + process_records
NG: POST /api/ng_process | LINE OUT: POST /api/lineout_process
IN-LINE: POST /api/in_line_process | REPAIRED: POST /api/stop_repaired_process

Step 4: Monitoring

Cycle Graph: GET /api/get_cycle_graph_data -> reads process_{N} tables
Line Trend: GET /api/get_line_trend_data -> reads process_records

3. Database Tables & Purpose

Database: cycle_time_monitoring (MySQL via XAMPP)

Cycle Time Tables

#	Table	Purpose
1	process_records	Master log of all cycle time records across all processes. Stores kitting_no, elapsed_time, pass/ng, lineout/inline reasons, timestamps.
2	process_1 ... process_9	Per-process logs - same data separated by process for faster queries. Includes operator_name, time_in, time_out, out_reasons.
3	standard_times	Standard time config - target cycle time (minutes) and title per process (e.g., Process 1 = "EM Fastening", ST = 1.56 min).
4	manpower	Operator assignments - current operator per process (id_no, name, employment_status, scan data, time_in). One row per process (1-9).

Material Setter Tables

5	MAIN_DB	Current material state - scanned materials for active job order with remaining quantities. Unique per (job_order, row_no). Updated on each scan.
6	KITTING_DB	Kitting history - every completed kitting creates records here. Stores kitting_no, material details, QR code data. Used by process tablets to validate QR scans.
7	kitting_summary	Live kitting summary - mirrors current kitting state in Material Setter UI. Unique per (job_order, row_no).
8	kitting_material	Material categories - 23 predefined categories (Frame Block, EM Block, Spacer, etc.) used as row definitions in the kitting table.
9	joborder_plan	Kitting progress tracker - plan/result/balance counts per job order. Also stores operator info, time_in/out, change_model_reason.

Operator & Reason Tables

10	mtrl_set_operator	Material Setter operator - single-row table for currently signed-in MS operator (id_no, name, time_in).
11	ms_operator_selection	Display preference - stores whether MS operator display shows "id", "name", or "all".
12	bio_break	OUT reasons - predefined reasons for operator OUT (CR, CLINIC, GO TO OTHER LINE, EMERGENCY, SENT HOME).
13	lineout_reasons	LINE OUT reasons - predefined reasons for line-out events (NG PRESSURE, LEAK, LW, LAV, etc.).
14	in_line_reasons	IN-LINE reasons - predefined reasons for returning in-line after a line-out.
15	repaired_actions	Repaired actions - predefined repair action descriptions for NG items that are fixed.
16	ms_change_model_reasons	Change model reasons - predefined reasons (PERFORMANCE PROBLEM, LACKING MATLS, MACHINE PROBLEM).

26 Material-Specific Tables

Each kitting completion saves per-row detail into the corresponding material table for traceability.

All 26 tables share the same schema: id, kitting_no, job_order, model_code, mtrl_desc, qty_unit, scan_mtrl, lot_no, qty_kit, kitting_qr_code, date_today, timestamp

#	Table	Material	#	Table	Material
01	01tbl_frame_zam	Frame Block	14	14tbl_ground_wire	Ground Wire
02	02tbl_frame_lead	EM Block	15	15tbl_vcr	VCR
03	03tbl_spacer	Spacer	16	16tbl_lower_housing	Lower Housing
04	04tbl_2p	DF Block	17	17tbl_rubber_foot	Rubber Foot
05	05tbl_p3p	ROD Block	18	18tbl_power_cord	Powercord
06	06tbl_df_block	Casing Block	19	19tbl_partition_gasket	Part. Gasket
07	07tbl_rod	Partition Board	20	20tbl_upper_housing	Upper Housing
08	08tbl_casing_block	L-Tube	21	21tbl_filter_cover	Filter Cover
09	09tbl_partition_board	Hose Clip	22	22tbl_filter	Filter
10	10tbl_l_tube	Switch Block	23	23tbl_sound_absorbing	Sound Absorber
11	11tbl_hose_clip	Lead Bushing	24	24tbl_manual	Manual
12	12tbl_switch_block	Ground Wire	25	25tbl_l_hose_set	L-Hose Set
13	13tbl_lead_bushing	VCR	26	26tbl_filter_bush	Filter Bushing

4. API Endpoint Map

Pages (GET -> HTML)

#	Route	Page
1	/	Main Menu
2	/tablet_menu	Tablet Main Menu
3	/material_setter	Material Setter (Desktop)
4	/material_setter_tablet	Material Setter (Tablet)
5	/cycle_time_home	Cycle Time Home
6	/process1 ... /process9	Process 1-9 Monitoring
7	/settings	Settings
8	/standard_time	Standard Time Configuration
9	/manpower	Manpower Configuration
10	/cycle_graph	Cycle Time Graph
11	/line_trend	Line Trend Graph

Arduino & Interlock APIs

Method	Endpoint	Purpose
POST	/api/arduino_signal	Receive cycle time button signals from Arduino bridge
GET	/api/arduino_signal/<process_no>	Check pending Arduino signal for a process
POST	/api/ms_arduino_signal	Receive MS sensor signals (MS1LOAD, MS1UNLOAD, P1START_MS, interlock)
GET	/api/ms_button_status	Browser polls MS button/interlock state
GET	/api/ms_get_pending_command	Bridge polls for pending relay commands (RELAY2_ON/OFF)
POST	/api/ms_relay2_control	Manual relay control (action: on/off)
POST	/api/bridge_ready	Bridge reports ready status

Cycle Time Process APIs

POST	/api/start_process	Start timer for a process
POST	/api/stop_process	Stop timer (PASS)
POST	/api/ng_process	Stop timer (NG)
POST	/api/lineout_process	Line out with reason
POST	/api/in_line_process	Return in-line
POST	/api/stop_repaired_process	Stop repaired NG item
GET	/api/get_process_records/<no>	Get records for a process
GET	/api/get_process_state/<no>	Get counter + timer state

Server Timer & Counter APIs

Method	Endpoint	Purpose
GET	/api/get_server_timer/<no>	Get server-side timer state
POST	/api/start_server_timer/<no>	Start server timer
POST	/api/stop_server_timer/<no>	Stop server timer
POST	/api/clear_server_timer/<no>	Clear server timer
POST	/api/sync_counter/<no>	Sync browser counter with server
GET	/api/get_completed_count/<no>	DB completed count
GET	/api/can_start_kitting/<no>/<kn>	Check if kitting can start
POST	/api/block_counter	Block a counter
POST	/api/reset_all	Reset all counters and timers

Material Setter & Kitting APIs

Method	Endpoint	Purpose
POST	/api/save_main_db	Save materials to MAIN_DB
GET	/api/get_main_db	Get materials for a job order
POST	/api/clear_main_db	Clear MAIN_DB for a job order
POST	/api/complete_kitting	Complete kitting -> KITTING_DB + material tables + RELAY2_ON
GET	/api/get_kitting_db	Get kitting records
POST	/api/validate_kitting_qr	Validate scanned kitting QR at process tablets
GET	/api/get_kitting_summary	Get kitting summary
GET	/api/get_next_kitting_no	Get next kitting number
POST	/api/print_kitting_qr	Print kitting QR code via Zebra printer

Operator APIs

Method	Endpoint	Purpose
POST	/api/ms_operator_in	MS operator sign in
POST	/api/ms_operator_out	MS operator sign out
GET	/api/get_ms_operator	Get current MS operator
POST	/api/update_manpower	Update process operator
POST	/api/operator_out	Process operator sign out
GET	/api/get_manpower	Get all manpower
GET	/api/get_manpower/<no>	Get operator for a process

Settings & Config APIs

Method	Endpoint	Purpose
GET	/api/get_settings	Get all settings
GET	/api/get_standard_times	Get standard times
POST	/api/save_printer_selection	Save printer config
GET	/api/get_model_codes	Get model codes
POST	/api/add_model_code	Add model code
GET	/api/get_kitting_materials	Get material categories
POST	/api/export_csv_now	Export data to CSV

Monitoring APIs

Method	Endpoint	Purpose
GET	/api/get_cycle_graph_data	Cycle time graph data

GET	/api/get_line_trend_data	Line trend data
GET	/api/get_all_active_timers	All active timer states
GET	/api/server_start_time	Server start time (auto-reload)

5. File Map

File	Role
cycleTimeMoni.py	Flask web server - all routes and API endpoints
database_manager.py	MySQL database operations - all CRUD functions
torisql.py	Toritani external database connector
qr_printer.py	Zebra printer control (ZPL/CPCL)
arduino_bridge.py	Serial bridge for Cycle Time Arduino (read-only)
arduino_bridge_ms.py	Serial bridge for Material Setter Arduino (bidirectional)
arduino_ms/arduino_ms.ino	Arduino UNO sketch for Material Setter (sensors + relay)
arduino_cycle_time/	Arduino sketch for Cycle Time buttons
templates/	20 HTML templates (process1-9, material_setter, settings, etc.)
static/	CSS, JS, images
material_setter_settings.json	MS config (printer, row count, display options)
model_codes_no_bushing.json	Model codes that skip filter bushing row
kitting_materials.json	Material category definitions
server_timer_state.json	Persisted timer/counter state across restarts